

Tobit (intreg-style) vs Beta-binomial: Diagnostics & Model Comparison

Contents

1) Data and shared preprocessing	1
2) Tobit / intreg-style model (Stata-replication likelihood)	2
2.1 Fit model	2
2.2 Tobit diagnostics (fit, censoring calibration, surprises)	4
2.3 Tobit diagnostic plots	6
3) Beta-binomial model (glmmTMB) on reconstructed counts	7
3.1 Reconstruct integer counts	7
3.2 Fit beta-binomial and run diagnostics	8
3.3 Cluster-robust inference (beta-binomial)	11
4) Common-scale predictive checks (Tobit vs Beta-binomial)	12
4.1 Predictions: mean share and P(zero)	12
4.2 Overall predictive check table	13
4.3 Predictive checks by period	13
4.4 Calibration plots by period (clearly titled)	14
4.5 Zero calibration by deciles	16
5) Tobit vs Beta-binomial: direction + significance table (rounded)	16
Adon: Sensitivity Threshold in Tobit model	18

1) Data and shared preprocessing

```
# Update path if needed
# df_raw <- read_excel("S1_file_combined.xlsx")
df_raw <- read_excel("S1_file_combined.xlsx")

# Analysis dataset (Model 3 covariates)
dat <- df_raw %>%
  transmute(
    tr_nop = as.numeric(tr_nop),
    ncases = as.numeric(ncases),
    Region_Period_Cluster_Unit = as.factor(co2LF),
    Region = as.factor(co2),

    # period indicators (unchanged)
    Neolithic = as.numeric(`T0n`),
    Early_Bronze_Age = as.numeric(T2be),
    Middle_Bronze_Age = as.numeric(T3bm),
    Late_Bronze_Age = as.numeric(T4b1),
    Iron_Age = as.numeric(`T5i`),

    # Model 3 covariates
    Battle_Sites = as.numeric(battle),
```

```

    Royal_Interments = as.numeric(royal),
    Urban = as.numeric(urban)
  ) %>%
  filter(
    !is.na(tr_nop), !is.na(ncases), !is.na(Region_Period_Cluster_Unit), !is.na(Region),
    !is.na(Neolithic), !is.na(Early_Bronze_Age), !is.na(Middle_Bronze_Age), !is.na(Late_Bronze_Age), !is.na(Battle_Sites), !is.na(Royal_Interments), !is.na(Urban),
    ncases > 0
  )

stopifnot(nrow(dat) == 82)

# Match Stata base category if relevant
if ("ir" %in% levels(dat$Region)) dat$Region <- relevel(dat$Region, ref = "ir")

# Period labels (used later)
dat <- dat %>%
  mutate(
    period = case_when(
      Neolithic == 1 ~ "Neolithic",
      Early_Bronze_Age == 1 ~ "Early_Bronze_Age",
      Middle_Bronze_Age == 1 ~ "Middle_Bronze_Age",
      Late_Bronze_Age == 1 ~ "Late_Bronze_Age",
      Iron_Age == 1 ~ "Iron_Age",
      TRUE ~ "BASE"
    )
  )

# Shared design matrix for Tobit/intreg model
X <- model.matrix(
  ~ Neolithic + Early_Bronze_Age + Middle_Bronze_Age + Late_Bronze_Age + Iron_Age + Battle_Sites + Royal_Interments + Urban,
  data = dat
)
K <- ncol(X)
N <- nrow(dat)

cluster <- dat$Region_Period_Cluster_Unit

```

2) Tobit / intreg-style model (Stata-replication likelihood)

2.1 Fit model

```

# Tobit/intreg outcome construction (unchanged)
y <- ifelse(dat$tr_nop > 0, log(dat$tr_nop), NA_real_)
is_cens <- dat$tr_nop == 0

# Stata-normalized analytic weights: sum(a) = N (unchanged)
a <- dat$ncases
stopifnot(all(is.finite(a)), all(a > 0))
a <- a * N / sum(a)

# Left-censor point (baseline)
cpoint <- -3

```

```

loglik <- function(par) {
  beta <- par[1:K]
  lns <- par[K + 1]
  s <- exp(lns)
  xb <- as.vector(X %*% beta)

  ll <- numeric(N)

  # Uncensored
  idx_u <- which(!is_cens)
  r <- y[idx_u] - xb[idx_u]
  ll[idx_u] <- -0.5 * ( a[idx_u] * (r^2) / (s^2) + log(2*pi) + 2*lns - log(a[idx_u]) )

  # Left-censored
  idx_c <- which(is_cens)
  t <- (cpoint - xb[idx_c]) * sqrt(a[idx_c]) / s
  ll[idx_c] <- pnorm(t, log.p = TRUE)

  sum(ll)
}

start <- c(rep(0, K), 0)
fit <- optim(start, fn = function(p) -loglik(p), method = "BFGS", hessian = TRUE)
stopifnot(fit$convergence == 0)

theta <- fit$par
beta_hat <- theta[1:K]
lnsigma_hat <- theta[K + 1]
sigma_hat <- exp(lnsigma_hat)

xb <- as.vector(X %*% beta_hat)

# Per-observation scores U for cluster-robust sandwich
U <- matrix(0, nrow = N, ncol = K + 1)
colnames(U) <- c(colnames(X), "lnsigma")

# Uncensored scores
idx_u <- which(!is_cens)
r <- y[idx_u] - xb[idx_u]
U[idx_u, 1:K] <- (a[idx_u] * r / sigma_hat^2) * X[idx_u, , drop = FALSE]
U[idx_u, K + 1] <- -1 + (a[idx_u] * r^2 / sigma_hat^2)

# Censored scores
idx_c <- which(is_cens)
tc <- (cpoint - xb[idx_c]) * sqrt(a[idx_c]) / sigma_hat
log_phi <- dnorm(tc, log = TRUE)
log_Phi <- pnorm(tc, log.p = TRUE)
lambda <- exp(pmin(log_phi - log_Phi, 700))

U[idx_c, 1:K] <- -(lambda * sqrt(a[idx_c]) / sigma_hat) * X[idx_c, , drop = FALSE]
U[idx_c, K + 1] <- -(lambda * tc)

# Cluster-robust sandwich

```

```

bread <- solve(fit$hessian)
Sg <- rowsum(U, cluster)
meat <- crossprod(Sg)
V_CR0 <- bread %*% meat %*% bread

G <- nrow(Sg)
V_stata <- (G/(G - 1)) * V_CR0 # Stata-style small-sample correction

Vb <- V_stata[1:K, 1:K, drop = FALSE]
se <- sqrt(diag(Vb))
zv <- beta_hat / se
pv <- 2 * pnorm(-abs(zv))

out_tobit <- data.frame(
  term = colnames(X),
  estimate = beta_hat,
  se_cr = se,
  z = zv,
  p = pv
)

cat("Tobit/intreg baseline fit complete.\n")

## Tobit/intreg baseline fit complete.

round_df(out_tobit, 3)

```

```

##           term estimate se_cr      z      p
## 1      (Intercept)  -1.940 0.451 -4.297 0.000
## 2         Neolithic  -1.332 0.308 -4.326 0.000
## 3   Early_Bronze_Age  -0.800 0.249 -3.219 0.001
## 4   Middle_Bronze_Age -1.815 0.712 -2.551 0.011
## 5     Late_Bronze_Age  0.715 0.810  0.883 0.377
## 6         Iron_Age   -1.113 0.254 -4.379 0.000
## 7     Battle_Sites    1.157 0.446  2.596 0.009
## 8   Royal_Interments  1.861 0.469  3.967 0.000
## 9         Urban      0.551 0.429  1.285 0.199
## 10        Regionl_   -0.515 0.371 -1.387 0.165
## 11        Regionms  -1.616 0.577 -2.802 0.005
## 12        Regiontr  -0.562 0.311 -1.809 0.070

```

2.2 Tobit diagnostics (fit, censoring calibration, surprises)

```

ll_hat <- loglik(theta)
k_par <- K + 1
AIC_t <- -2 * ll_hat + 2 * k_par
BIC_t <- -2 * ll_hat + log(N) * k_par

# Wald chi^2 for slopes (robust)
idx_slope <- which(colnames(X) != "(Intercept)")
b_slope <- beta_hat[idx_slope]
V_slope <- Vb[idx_slope, idx_slope, drop = FALSE]
Wald_chi2 <- as.numeric(t(b_slope) %*% solve(V_slope) %*% b_slope)
df_wald <- length(idx_slope)

```

```

p_wald <- pchisq(Wald_chi2, df = df_wald, lower.tail = FALSE)

# Model-implied P(censored)
sd_i <- sigma_hat / sqrt(a)
p_cens_hat <- pnorm((cpoint - xb) / sd_i)

# Observed vs predicted censoring rates
obs_cens_unw <- mean(is_cens)
pred_cens_unw <- mean(p_cens_hat)
obs_cens_w <- weighted.mean(is_cens, w = a)
pred_cens_w <- weighted.mean(p_cens_hat, w = a)

# Brier score for censoring indicator
brier_w <- weighted.mean((as.numeric(is_cens) - p_cens_hat)^2, w = a)

# RMSE among uncensored (descriptive)
res_u <- y[!is_cens] - xb[!is_cens]
rmse_u <- sqrt(weighted.mean(res_u^2, w = a[!is_cens]))

# Null model for McFadden pseudo-R2
loglik0 <- function(par0) {
  b0 <- par0[1]
  lns0 <- par0[2]
  s0 <- exp(lns0)
  xb0 <- rep(b0, N)

  l10 <- numeric(N)
  idx_u0 <- which(!is_cens)
  r0 <- y[idx_u0] - xb0[idx_u0]
  l10[idx_u0] <- -0.5 * ( a[idx_u0] * (r0^2) / (s0^2) + log(2*pi) + 2*lns0 - log(a[idx_u0]) )

  idx_c0 <- which(is_cens)
  t0 <- (cpoint - xb0[idx_c0]) * sqrt(a[idx_c0]) / s0
  l10[idx_c0] <- pnorm(t0, log.p = TRUE)

  sum(l10)
}

fit0 <- optim(c(0, 0), fn = function(p) -loglik0(p), method = "BFGS", hessian = TRUE)
stopifnot(fit0$convergence == 0)
ll_null <- loglik0(fit0$par)
pseudoR2 <- 1 - (ll_hat / ll_null)

# Surprise ranking (large values = surprising given model)
p_unc_hat <- 1 - p_cens_hat
surprise <- ifelse(is_cens, -log(p_cens_hat), -log(p_unc_hat))
w_surprise <- a * surprise
ord <- order(w_surprise, decreasing = TRUE)

fit_stats_tobit <- data.frame(
  N = N,
  uncensored = sum(!is_cens),
  left_censored = sum(is_cens),

```

```

clusters = G,
logLik = ll_hat,
AIC = AIC_t,
BIC = BIC_t,
Wald_chi2 = Wald_chi2,
df = df_wald,
p_value = p_wald,
obs_cens_unw = obs_cens_unw,
pred_cens_unw = pred_cens_unw,
obs_cens_w = obs_cens_w,
pred_cens_w = pred_cens_w,
brier_w = brier_w,
rmse_uncensored = rmse_u,
sigma = sigma_hat,
ll_null = ll_null,
pseudoR2_mcfadden = pseudoR2,
max_w_surprise = max(w_surprise)
)

round_df(fit_stats_tobit, 3)

##      N uncensored left_censored clusters   logLik      AIC BIC Wald_chi2 df
## 1 82          48          34          21 -104.356 234.713 266   538.442 11
##   p_value obs_cens_unw pred_cens_unw obs_cens_w pred_cens_w brier_w
## 1      0      0.415      0.553      0.125      0.6    0.431
##   rmse_uncensored sigma  ll_null pseudoR2_mcfadden max_w_surprise
## 1      0.932 1.119 -120.087      0.131      88.319

top_surprise <- data.frame(
  row = ord[1:10],
  weight_a = a[ord[1:10]],
  xb = xb[ord[1:10]],
  is_cens = is_cens[ord[1:10]],
  p_cens = p_cens_hat[ord[1:10]],
  w_surprise = w_surprise[ord[1:10]]
)

round_df(top_surprise, 3)

##      row weight_a      xb is_cens p_cens w_surprise
## 1    20  14.505 -3.834  FALSE  0.998    88.319
## 2    45   8.411 -3.302  FALSE  0.783    12.849
## 3    12   3.985 -3.255  FALSE  0.675     4.481
## 4    24   2.201 -3.834  FALSE  0.866     4.417
## 5    16   3.082 -3.302  FALSE  0.682     3.531
## 6    33   2.456 -3.302  FALSE  0.664     2.677
## 7    19   2.178 -3.283  FALSE  0.646     2.259
## 8     8   1.112 -3.834  FALSE  0.784     1.705
## 9    13   1.112 -3.834  FALSE  0.784     1.705
## 10    4   1.622 -3.283  FALSE  0.626     1.597

```

2.3 Tobit diagnostic plots

```

op <- par(no.readonly = TRUE)
par(mfrow = c(1, 2))

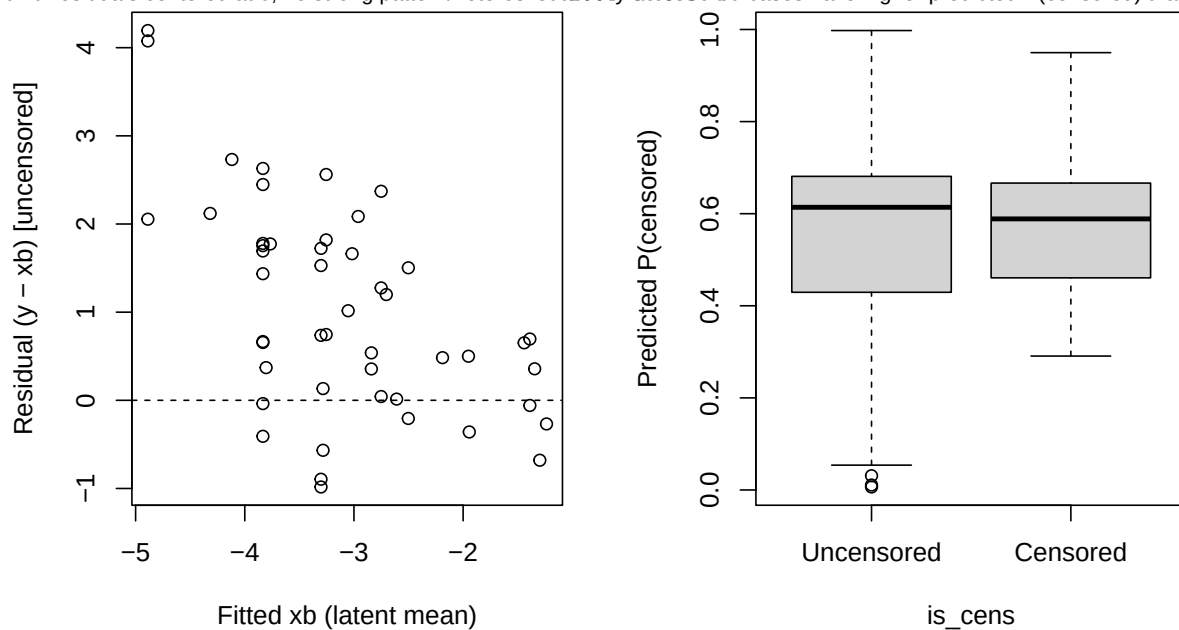
plot(
  xb[!is_cens], res_u,
  xlab = "Fitted xb (latent mean)",
  ylab = "Residual (y - xb) [uncensored]",
  main = sprintf("Tobit/intreg (cpoint=%s): Residuals vs fitted", cpoint)
)
abline(h = 0, lty = 2)
add_goodfit_subtitle("Good fit: residuals centered at 0; no strong pattern/heteroskedasticity across xb")

boxplot(
  p_cens_hat ~ is_cens,
  names = c("Uncensored", "Censored"),
  ylab = "Predicted P(censored)",
  main = sprintf("Tobit/intreg (cpoint=%s): P(censored) by observed status", cpoint)
)
add_goodfit_subtitle("Good fit: censored cases have higher predicted P(censored) than uncensored cases.")

```

Tobit/intreg (cpoint=-3): Residuals vs fitted/intreg (cpoint=-3): P(censored) by observed status

Good fit: residuals centered at 0; no strong pattern/heteroskedasticity across xb
 Good fit: censored cases have higher predicted P(censored) than uncensored cases.



```
par(op)
```

3) Beta-binomial model (glmmTMB) on reconstructed counts

3.1 Reconstruct integer counts

```

dat2 <- dat %>%
  select(period, everything()) %>% # <-- ensure period is present
  mutate(
    n = as.integer(round(ncases)),
    p = tr_nop,

```

```

    k_raw = p * n,
    k = as.integer(round(k_raw)),
    k = pmin(pmax(k, 0L), n)
  )

stopifnot("period" %in% names(dat2))
stopifnot(all(dat2$n > 0), all(dat2$p >= 0 & dat2$p <= 1))

recon_check <- dat2 %>%
  mutate(diff = k_raw - k) %>%
  summarise(
    max_abs_diff = max(abs(diff)),
    mean_abs_diff = mean(abs(diff)),
    share_exact_integer = mean(abs(diff) < 1e-8)
  )

round_df(recon_check, 3)

## # A tibble: 1 x 3
##   max_abs_diff mean_abs_diff share_exact_integer
##         <dbl>         <dbl>         <dbl>
## 1         0.5         0.021         0.512

```

3.2 Fit beta-binomial and run diagnostics

```

if (!have_pkg("glmmTMB")) stop("Please install glmmTMB")
library(glmmTMB)

f_bin <- cbind(k, n - k) ~ Neolithic + Early_Bronze_Age + Middle_Bronze_Age + Late_Bronze_Age + Iron_Age

m_bb <- glmmTMB(
  f_bin,
  data = dat2,
  family = betabinomial(link = "logit")
)

summary(m_bb)

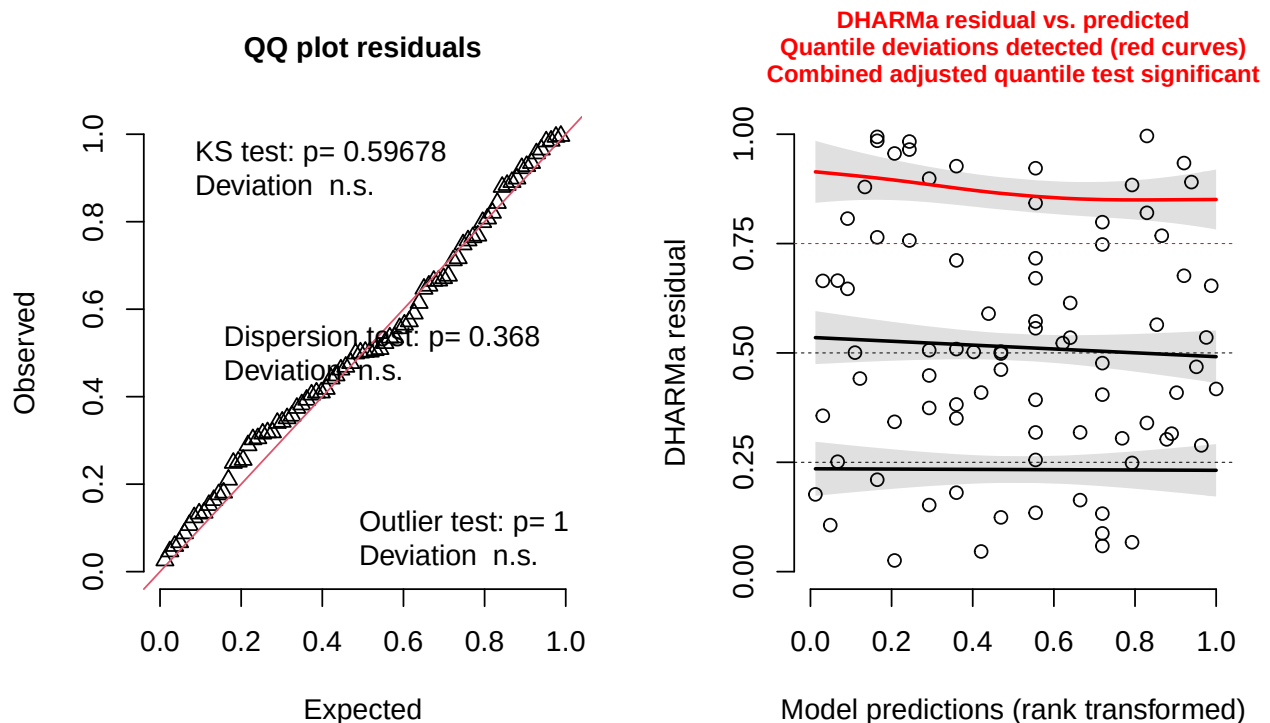
## Family: betabinomial ( logit )
## Formula:
## cbind(k, n - k) ~ Neolithic + Early_Bronze_Age + Middle_Bronze_Age +
##   Late_Bronze_Age + Iron_Age + Battle_Sites + Royal_Interments +
##   Urban + Region
## Data: dat2
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    329.5    360.8   -151.8    303.5        69
##
##
## Dispersion parameter for betabinomial family (): 7.96
##
## Conditional model:
##               Estimate Std. Error z value Pr(>|z|)

```

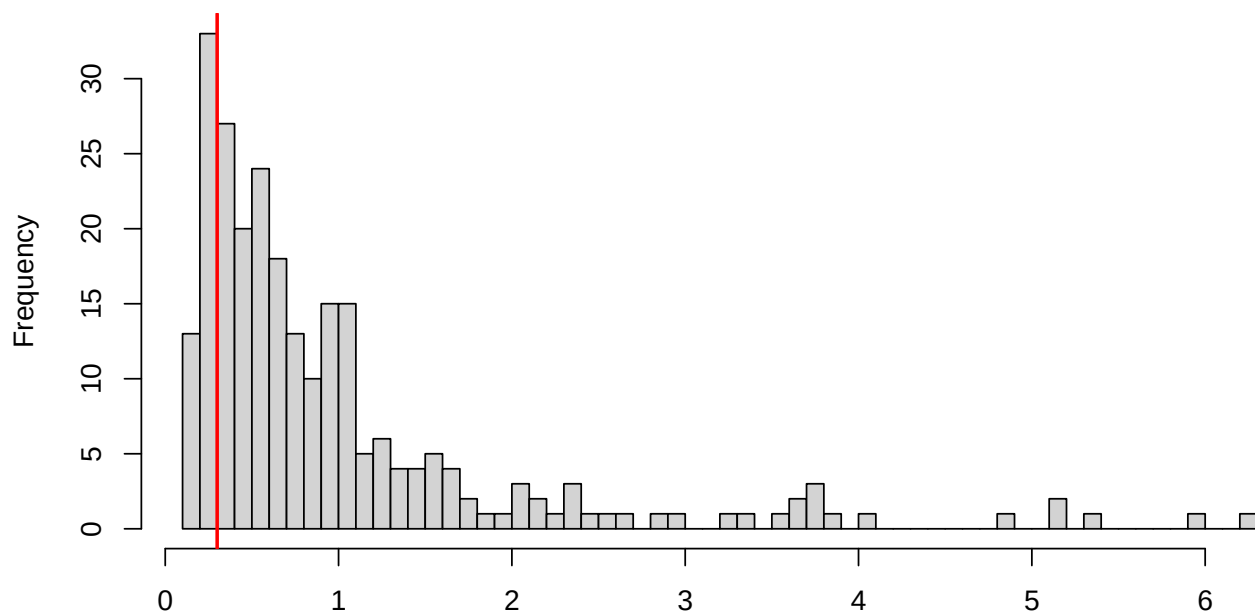
```
## (Intercept)      -1.54415    0.57166   -2.701    0.00691 **
## Neolithic        -0.30315    0.48656   -0.623    0.53326
## Early_Bronze_Age -0.07668    0.47396   -0.162    0.87148
## Middle_Bronze_Age -1.27811    0.73319   -1.743    0.08130 .
## Late_Bronze_Age   0.64518    0.70012    0.921    0.35678
## Iron_Age          -0.24179    0.57540   -0.420    0.67433
## Battle_Sites      1.41407    0.52202    2.709    0.00675 **
## Royal_Interments  1.30846    0.69011    1.896    0.05796 .
## Urban             0.09604    0.31367    0.306    0.75947
## Regionl_          -1.25889    0.58723   -2.144    0.03205 *
## Regionms          -1.13199    0.55872   -2.026    0.04276 *
## Regiontr          -0.26760    0.53935   -0.496    0.61979
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# DHARMA residual diagnostics (titled)
if (have_pkg("DHARMA")) {
  library(DHARMA)
  res_bb <- simulateResiduals(m_bb, plot = FALSE)
  plot(res_bb)
  print(testDispersion(res_bb))
  print(testZeroInflation(res_bb))
}
```

DHARMA residual



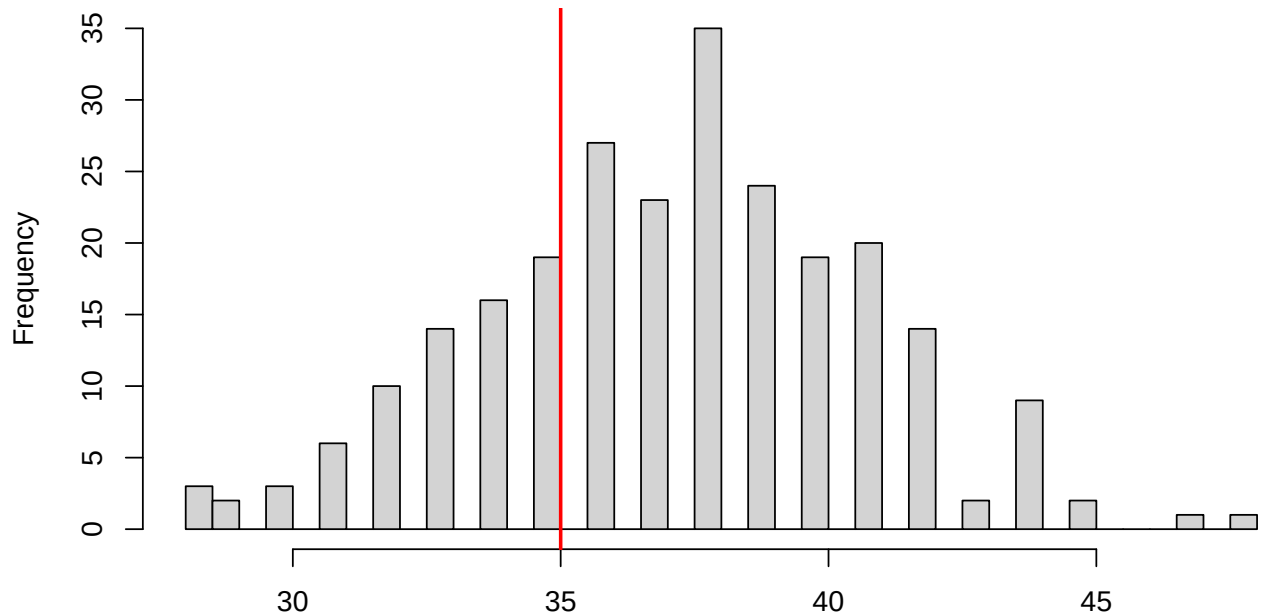
DHARMA nonparametric dispersion test via sd of
residuals fitted vs. simulated



Simulated values, red line = fitted model. p-value (two.sided) = 0.368

```
##  
## DHARMA nonparametric dispersion test via sd of residuals fitted vs.  
## simulated  
##  
## data: simulationOutput  
## dispersion = 0.29846, p-value = 0.368  
## alternative hypothesis: two.sided
```

DHARMA zero-inflation test via comparison to expected zeros with simulation under H0 = fitted model



Simulated values, red line = fitted model. p-value (two.sided) = 0.584

```
##
## DHARMA zero-inflation test via comparison to expected zeros with
## simulation under H0 = fitted model
##
## data: simulationOutput
## ratioObsSim = 0.93753, p-value = 0.584
## alternative hypothesis: two.sided
```

3.3 Cluster-robust inference (beta-binomial)

```
# Cluster-robust HCO (sandwich) from glmmTMB
V_hc0 <- glmmTMB::vcovHC(m_bb, cluster = dat2$Region_Period_Cluster_Unit) # HCO only

# Stata-style small-sample multiplier you used in Tobit
G <- dplyr::n_distinct(dat2$Region_Period_Cluster_Unit)
V_stata <- (G/(G - 1)) * V_hc0

# Fixed effects (conditional) inference using z
b <- glmmTMB::fixef(m_bb)$cond
Vb <- V_stata[names(b), names(b), drop = FALSE]

se <- sqrt(diag(Vb))
z <- b / se
p <- 2 * pnorm(-abs(z))

bb_inf <- data.frame(term = names(b), estimate = b, se_cr = se, z = z, p = p)
bb_inf
```

	term	estimate	se_cr	z
## (Intercept)	(Intercept)	-1.54414900	0.7301176	-2.1149319
## Neolithic	Neolithic	-0.30314576	0.4725623	-0.6414937
## Early_Bronze_Age	Early_Bronze_Age	-0.07667713	0.4160641	-0.1842917
## Middle_Bronze_Age	Middle_Bronze_Age	-1.27810761	0.6237108	-2.0491991
## Late_Bronze_Age	Late_Bronze_Age	0.64517764	0.4345091	1.4848428
## Iron_Age	Iron_Age	-0.24179389	0.5548115	-0.4358127
## Battle_Sites	Battle_Sites	1.41407433	0.4618118	3.0620145
## Royal_Interments	Royal_Interments	1.30846044	0.2566208	5.0988096
## Urban	Urban	0.09603614	0.3904362	0.2459714
## Regionl_	Regionl_	-1.25888906	0.5818661	-2.1635374
## Regionms	Regionms	-1.13199193	0.5807312	-1.9492530
## Regiontr	Regiontr	-0.26759748	0.5960344	-0.4489632

	p
## (Intercept)	0.0344357489075
## Neolithic	0.5212019625915
## Early_Bronze_Age	0.8537846630213
## Middle_Bronze_Age	0.0404426480741
## Late_Bronze_Age	0.1375854838971
## Iron_Age	0.6629726091267
## Battle_Sites	0.0021985281669
## Royal_Interments	0.0000003417961
## Urban	0.8057043560371
## Regionl_	0.0304998718453
## Regionms	0.0512652238068
## Regiontr	0.6534582351777

4) Common-scale predictive checks (Tobit vs Beta-binomial)

4.1 Predictions: mean share and P(zero)

```
# --- Tobit predictions on original SHARE scale via simulation ---
set.seed(1)
S <- 5000
sd_i <- sigma_hat / sqrt(a)
ystar <- matrix(rnorm(N * S), nrow = N, ncol = S)
ystar <- xb + sd_i * ystar

share_sim <- ifelse(ystar <= cpoint, 0, exp(ystar))
share_hat_tobit <- rowMeans(share_sim)
p0_hat_tobit <- rowMeans(ystar <= cpoint)

# --- Beta-binomial predictions ---
# Mean share (conditional mean) from glmmTMB
p_hat_bb <- as.numeric(predict(m_bb, type = "response"))

# P(k=0) via simulation
set.seed(1)
sim_obj <- simulate(m_bb, nsim = 500)
# simulate(m_bb) returns a data.frame whose columns contain N x 2 matrices [k, n-k]
sim_k_mat <- do.call(cbind, lapply(sim_obj, function(m) m[, 1]))
stopifnot(nrow(sim_k_mat) == N, ncol(sim_k_mat) == 500)
p0_hat_bb <- rowMeans(sim_k_mat == 0)
```

```

# Attach predictions to dat2 for safe grouped summaries
# (This prevents the classic weighted.mean length mismatch.)
dat_pred <- dat2 %>%
  mutate(
    tobit_share_hat = share_hat_tobit,
    tobit_p0_hat = p0_hat_tobit,
    bb_share_hat = p_hat_bb,
    bb_p0_hat = p0_hat_bb
  )

```

4.2 Overall predictive check table

```

obs_share <- dat_pred$p
obs_zero <- (dat_pred$k == 0)

summ_overall <- data.frame(
  metric = c("mean_share_unw", "mean_share_w_n",
             "zero_rate_unw", "zero_rate_w_n"),
  observed = c(
    mean(obs_share),
    weighted.mean(obs_share, w = dat_pred$n),
    mean(obs_zero),
    weighted.mean(obs_zero, w = dat_pred$n)
  ),
  tobit = c(
    mean(dat_pred$tobit_share_hat),
    weighted.mean(dat_pred$tobit_share_hat, w = dat_pred$n),
    mean(dat_pred$tobit_p0_hat),
    weighted.mean(dat_pred$tobit_p0_hat, w = dat_pred$n)
  ),
  betabinomial = c(
    mean(dat_pred$bb_share_hat),
    weighted.mean(dat_pred$bb_share_hat, w = dat_pred$n),
    mean(dat_pred$bb_p0_hat),
    weighted.mean(dat_pred$bb_p0_hat, w = dat_pred$n)
  )
)

round_df(as.data.frame(summ_overall), 3)

```

##		metric	observed	tobit	betabinomial
## 1	mean_share_unw		0.112	100301.533	0.102
## 2	mean_share_w_n		0.091	2330.163	0.121
## 3	zero_rate_unw		0.427	0.552	0.449
## 4	zero_rate_w_n		0.126	0.599	0.164

4.3 Predictive checks by period

```

# IMPORTANT: we summarise on dat_pred so x and w are always the same length in each group.

by_period <- dat_pred %>%
  group_by(period) %>%
  summarise(

```

```

n_obs = n(),
obs_share_w_n = weighted.mean(p, w = n),
obs_zero_w_n = weighted.mean(k == 0, w = n),
tobit_share_w_n = weighted.mean(tobit_share_hat, w = n),
tobit_p0_w_n = weighted.mean(tobit_p0_hat, w = n),
bb_share_w_n = weighted.mean(bb_share_hat, w = n),
bb_p0_w_n = weighted.mean(bb_p0_hat, w = n)
)

round_df(as.data.frame(by_period), 3)

```

```

##           period n_obs obs_share_w_n obs_zero_w_n tobit_share_w_n
## 1           BASE    10         0.203         0.131           4.309
## 2 Early_Bronze_Age   27         0.087         0.091       5677.735
## 3           Iron_Age   13         0.135         0.231           1.371
## 4 Late_Bronze_Age    4         0.101         0.077           4.117
## 5 Middle_Bronze_Age  10         0.045         0.508       7896.452
## 6        Neolithic   18         0.035         0.067           0.409
##  tobit_p0_w_n bb_share_w_n bb_p0_w_n
## 1         0.073         0.161         0.062
## 2         0.659         0.112         0.170
## 3         0.344         0.163         0.244
## 4         0.409         0.135         0.268
## 5         0.647         0.075         0.518
## 6         0.870         0.105         0.121

```

4.4 Calibration plots by period (clearly titled)

```

op <- par(no.readonly = TRUE)
par(mfrow = c(2, 2))

# 1) Beta-binomial: zero rate
plot(
  by_period$obs_zero_w_n, by_period$bb_p0_w_n,
  xlab = "Observed zero rate (weighted by n)",
  ylab = "Predicted P(zero) (weighted by n)",
  main = "Beta-binomial (glmmTMB): Zero-rate calibration by period",
  pch = 19
)
add_goodfit_subtitle(good_line_txt)
abline(0, 1, lty = 2)
text(by_period$obs_zero_w_n, by_period$bb_p0_w_n, labels = by_period$period, pos = 4, cex = 0.8)

# 2) Tobit: zero rate
plot(
  by_period$obs_zero_w_n, by_period$tobit_p0_w_n,
  xlab = "Observed zero rate (weighted by n)",
  ylab = "Predicted P(zero) (weighted by n)",
  main = sprintf("Tobit/intreg (cpoint=%s): Zero-rate calibration by period", cpoint),
  pch = 19
)
add_goodfit_subtitle(good_line_txt)
abline(0, 1, lty = 2)

```

```

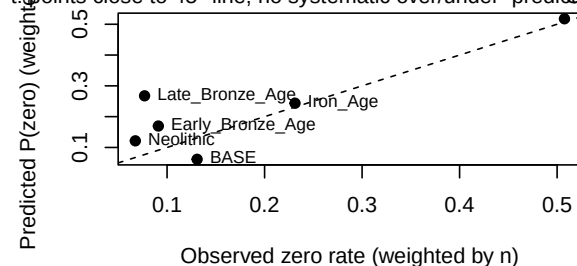
text(by_period$obs_zero_w_n, by_period$tobit_p0_w_n, labels = by_period$period, pos = 4, cex = 0.8)

# 3) Beta-binomial: mean share
plot(
  by_period$obs_share_w_n, by_period$bb_share_w_n,
  xlab = "Observed mean trauma share (weighted by n)",
  ylab = "Predicted mean trauma share (weighted by n)",
  main = "Beta-binomial (glmmTMB): Mean-share calibration by period",
  pch = 19
)
add_goodfit_subtitle("Good fit: points close to 45° line; errors not concentrated in specific periods.")
abline(0, 1, lty = 2)
text(by_period$obs_share_w_n, by_period$bb_share_w_n, labels = by_period$period, pos = 4, cex = 0.8)

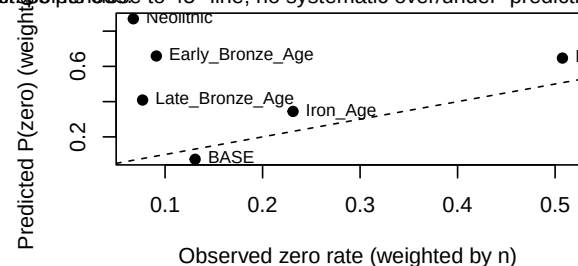
# 4) Tobit: mean share
plot(
  by_period$obs_share_w_n, by_period$tobit_share_w_n,
  xlab = "Observed mean trauma share (weighted by n)",
  ylab = "Predicted mean trauma share (weighted by n)",
  main = sprintf("Tobit/intreg (cpoint=%s): Mean-share calibration by period", cpoint),
  pch = 19
)
add_goodfit_subtitle("Good fit: points close to 45° line; no huge outliers (massive over-prediction).")
abline(0, 1, lty = 2)
text(by_period$obs_share_w_n, by_period$tobit_share_w_n, labels = by_period$period, pos = 4, cex = 0.8)

```

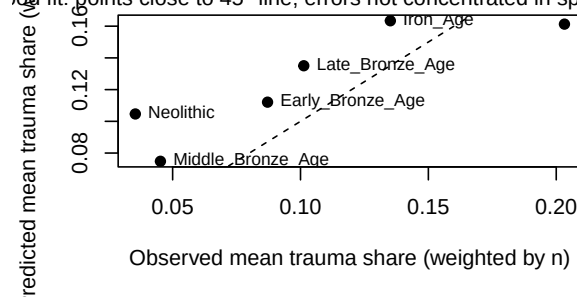
Beta-binomial (glmmTMB): Zero-rate calibration by period (Good fit: points close to 45° line; no systematic over/under-prediction across periods)



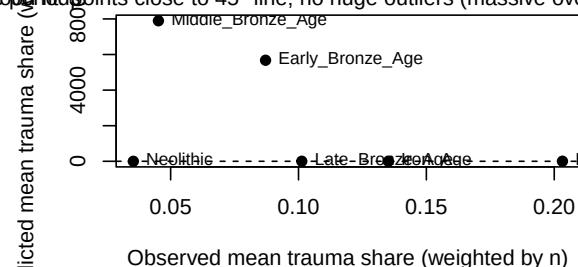
Tobit/intreg (cpoint=-3): Zero-rate calibration by period (Good fit: points close to 45° line; no systematic over/under-prediction across periods)



Beta-binomial (glmmTMB): Mean-share calibration by period (Good fit: points close to 45° line; errors not concentrated in specific periods)



Tobit/intreg (cpoint=-3): Mean-share calibration by period (Good fit: points close to 45° line; no huge outliers (massive over-prediction))



```
par(op)
```

4.5 Zero calibration by deciles

```
make_calibration <- function(p0_hat, obs_zero, w, name) {
  cuts <- unique(quantile(p0_hat, probs = seq(0, 1, 0.1), na.rm = TRUE))
  bin <- cut(p0_hat, breaks = cuts, include.lowest = TRUE)
  tibble(bin = bin, p0_hat = p0_hat, obs_zero = obs_zero, w = w) %>%
    group_by(bin) %>%
    summarise(
      n = n(),
      pred_w = weighted.mean(p0_hat, w, na.rm = TRUE),
      obs_w = weighted.mean(obs_zero, w, na.rm = TRUE),
      .groups = "drop"
    ) %>%
    mutate(model = name)
}

cal_all <- bind_rows(
  make_calibration(dat_pred$tobit_p0_hat, (dat_pred$k == 0), dat_pred$n, "tobit"),
  make_calibration(dat_pred$bb_p0_hat, (dat_pred$k == 0), dat_pred$n, "betabinomial")
)

round_df(as.data.frame(cal_all), 3)
```

##	bin	n	pred_w	obs_w	model
## 1	[0.0072,0.272]	9	0.068	0.000	tobit
## 2	(0.272,0.432]	8	0.360	0.381	tobit
## 3	(0.432,0.451]	8	0.444	0.383	tobit
## 4	(0.451,0.517]	8	0.495	0.218	tobit
## 5	(0.517,0.595]	8	0.570	0.714	tobit
## 6	(0.595,0.633]	8	0.619	0.126	tobit
## 7	(0.633,0.66]	8	0.650	0.267	tobit
## 8	(0.66,0.693]	8	0.673	0.053	tobit
## 9	(0.693,0.783]	8	0.768	0.035	tobit
## 10	(0.783,0.997]	9	0.955	0.130	tobit
## 11	[0,0.042]	10	0.023	0.000	betabinomial
## 12	(0.042,0.122]	7	0.090	0.090	betabinomial
## 13	(0.122,0.197]	8	0.164	0.000	betabinomial
## 14	(0.197,0.304]	8	0.259	0.103	betabinomial
## 15	(0.304,0.472]	8	0.406	0.078	betabinomial
## 16	(0.472,0.584]	9	0.544	0.686	betabinomial
## 17	(0.584,0.669]	7	0.635	0.770	betabinomial
## 18	(0.669,0.724]	8	0.704	0.907	betabinomial
## 19	(0.724,0.859]	8	0.777	0.686	betabinomial
## 20	(0.859,0.988]	9	0.884	0.905	betabinomial

5) Tobit vs Beta-binomial: direction + significance table (rounded)

```
# Tobit: already has cluster-robust p-values
# Beta-binomial: uses cluster-robust p-values if clubSandwich works, else model-based.

# Prepare Tobit table
term_t <- out_tobit$term
names(term_t) <- NULL
```

```

tobit_tbl <- out_tobit %>%
  transmute(
    term,
    est_tobit = estimate,
    p_tobit = p,
    sign_tobit = ifelse(estimate > 0, "+", "-")
  )

# Prepare beta-binomial table (cluster-bootstrap inference)
bb_tbl <- bb_inf %>%
  transmute(
    term,
    est_bb = estimate,
    p_bb = p,
    sign_bb = ifelse(estimate > 0, "+", "-")
  )

comp <- inner_join(tobit_tbl, bb_tbl, by = "term") %>%
  mutate(
    agree_sign = (sign_tobit == sign_bb),
    sig_tobit_05 = (p_tobit < 0.05),
    sig_bb_05 = (p_bb < 0.05)
  )

round_df(as.data.frame(comp), 3)

```

	term	est_tobit	p_tobit	sign_tobit	est_bb	p_bb	sign_bb
## 1	(Intercept)	-1.940	0.000	-	-1.544	0.034	-
## 2	Neolithic	-1.332	0.000	-	-0.303	0.521	-
## 3	Early_Bronze_Age	-0.800	0.001	-	-0.077	0.854	-
## 4	Middle_Bronze_Age	-1.815	0.011	-	-1.278	0.040	-
## 5	Late_Bronze_Age	0.715	0.377	+	0.645	0.138	+
## 6	Iron_Age	-1.113	0.000	-	-0.242	0.663	-
## 7	Battle_Sites	1.157	0.009	+	1.414	0.002	+
## 8	Royal_Interments	1.861	0.000	+	1.308	0.000	+
## 9	Urban	0.551	0.199	+	0.096	0.806	+
## 10	Regionl_	-0.515	0.165	-	-1.259	0.030	-
## 11	Regionms	-1.616	0.005	-	-1.132	0.051	-
## 12	Regiontr	-0.562	0.070	-	-0.268	0.653	-
##	agree_sign	sig_tobit_05	sig_bb_05				
## 1	TRUE	TRUE	TRUE				
## 2	TRUE	TRUE	FALSE				
## 3	TRUE	TRUE	FALSE				
## 4	TRUE	TRUE	TRUE				
## 5	TRUE	FALSE	FALSE				
## 6	TRUE	TRUE	FALSE				
## 7	TRUE	TRUE	TRUE				
## 8	TRUE	TRUE	TRUE				
## 9	TRUE	FALSE	FALSE				
## 10	TRUE	FALSE	TRUE				
## 11	TRUE	TRUE	FALSE				
## 12	TRUE	FALSE	FALSE				

Adon: Sensitivity Threshold in Tobit model

Table 1: Coefficients across censoring points (* p<0.05, ** p<0.01)

Variable

c=-5

c=-4.5

c=-4

c=-3.5

c=-3

c=-2.5

(Intercept)

-2.031**(0.451)

-2.004**(0.451)

-1.980**(0.451)

-1.958**(0.451)

-1.939**(0.451)

-1.925**(0.451)

Neolithic

-1.085**(0.308)

-1.150**(0.308)

-1.213**(0.308)

-1.274**(0.308)

-1.332**(0.308)

-1.386**(0.308)

Early_Bronze_Age

-0.438(0.249)

-0.534(0.249)

-0.628*(0.249)

-0.717**(0.249)

-0.800**(0.249)

-0.877**(0.249)

Middle_Bronze_Age

-2.572*(0.712)

-2.366*(0.712)

-2.170*(0.712)

-1.986*(0.712)

-1.815*(0.712)

-1.661*(0.712)
 Late_Bronze_Age
 1.886(0.810)
 1.567(0.810)
 1.263(0.810)
 0.978(0.810)
 0.715(0.810)
 0.479(0.810)
 Iron_Age
 -1.316**(0.254)
 -1.263**(0.254)
 -1.211**(0.254)
 -1.161**(0.254)
 -1.114**(0.254)
 -1.068**(0.254)
 Battle_Sites
 2.071**(0.446)
 1.824**(0.446)
 1.588**(0.446)
 1.365**(0.446)
 1.157**(0.446)
 0.968*(0.446)
 Royal_Interments
 2.632**(0.469)
 2.421**(0.469)
 2.220**(0.469)
 2.033**(0.469)
 1.861**(0.469)
 1.708**(0.469)
 Urban
 0.533(0.429)
 0.537(0.429)
 0.542(0.429)
 0.546(0.429)
 0.551(0.429)
 0.556(0.429)

Regionl_

-1.485**(0.371)

-1.226*(0.371)

-0.976*(0.371)

-0.738(0.371)

-0.515(0.371)

-0.309(0.371)

Regionms

-3.101**(0.577)

-2.698**(0.577)

-2.312**(0.577)

-1.950**(0.577)

-1.616**(0.577)

-1.313*(0.577)

Regiontr

-0.767(0.311)

-0.715(0.311)

-0.663(0.311)

-0.612(0.311)

-0.562(0.311)

-0.514(0.311)